

Mini Project Report
on
ML-Powered Medical Diagnosis Assistant

Submitted by
Team: Rachith Bharadwaj T N and team

Rachith Bharadwaj T N 24BDS062

Dhanush Gowda N 24BDS018

Shreedhar M Kadkol 24BDS076

Kishan Kumar Y 24BDS031

Under the guidance of
Dr. Utkarsh Mahadeo Khaire
Department of DSAI

Indian Institute of Information Technology Dharwad

Final Version: 14/06/2026

DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

14/06/2026

Certificate

This is to certify that the project, entitled **ML-Powered Medical Diagnosis Assistant**, is a bonafide record of the Mini Project coursework presented by the students whose names are given below during Academic Year 2024-25 in partial fulfilment of the requirements of the degree of Bachelor of Technology in Data Science and Artificial Intelligence.

Roll No	Names of Students
24BDS062	Rachith Bharadwaj T N
24BDS018	Dhanush Gowda N
24BDS076	Shreedhar M Kadkol
24BDS031	Kishan Kumar Y

Dr. Utkarsh Mahadeo Khairé
(Project Supervisor)

Contents

List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Background	7
1.2 Motivation	7
1.3 Objectives	7
2 Problem Statement	8
2.1 Healthcare Challenges in Resource-Limited Areas	8
2.2 Project Scope	8
3 Proposed Solution	9
3.1 System Overview	9
3.2 System Architecture	10
4 Technical Implementation	11
4.1 Technology Stack	11
4.2 Application Modules	11
5 Data and Methods	12
5.1 Dataset Description	12
5.2 Data Preprocessing	12
5.3 Model Training	13
6 Results and Discussions	14
6.1 Model Performance	14
6.2 Interface Results	14
7 System Features and Capabilities	16
8 Development and Testing	17
9 Limitations and Future Enhancements	18
10 Conclusion	19
References	20

List of Figures

1 System Architecture Diagram	10
2 Dataset Cleaning and Training Pipeline	13
3 Web Application Interface Workflow	15
4 Sample Diagnostic Report Output	15

List of Tables

1 Technology Stack Overview 11

2 Final Cleaned Dataset Summary 12

3 Model Performance Metrics 14

4 Completed and Planned Scope 18

1 Introduction

1.1 Background

Healthcare accessibility remains a major challenge in rural and resource-limited areas. A large number of patients first describe their condition through symptoms, but those symptoms are often unstructured, incomplete, or difficult to compare with standard disease patterns. Machine learning can help organize symptom descriptions into a preliminary ranked output for educational triage support.

The ML-Powered Medical Diagnosis Assistant is designed as an academic prototype that accepts symptoms through a web interface, maps recognizable symptom phrases to model features, and predicts likely disease labels using a trained TensorFlow classifier. The system is not a medical device and is not intended to replace doctors or qualified healthcare professionals.

1.2 Motivation

- Provide a simple local tool that demonstrates how machine learning can support preliminary symptom analysis.
- Make the project runnable from GitHub without depending on notebooks alone.
- Support offline or local-network classroom demonstration without ngrok or internet-loaded page assets.
- Present model outputs with confidence scores, matched evidence, urgency level, and recommended action.

1.3 Objectives

- Clean and align the available symptom-to-disease datasets for the final Flask application.
- Train a TensorFlow neural network classifier using binary symptom features.
- Build a web application for text and browser-supported voice symptom input.
- Generate ranked predictions and printable diagnosis reports.
- Document limitations clearly so the work is presented as an educational prototype.

2 Problem Statement

2.1 Healthcare Challenges in Resource-Limited Areas

Resource-limited settings often face delayed diagnosis, limited access to specialists, travel barriers, and poor connectivity. A lightweight local decision-support prototype can help demonstrate how symptoms may be structured before professional consultation.

2.1.1 Limited Access to Medical Professionals

Patients in remote areas may need to travel long distances for basic consultation. This can delay care and increase financial burden.

2.1.2 High Risk of Incomplete Symptom Reporting

Patients may describe symptoms in everyday language. A model-ready system must convert those descriptions into stable structured features.

2.1.3 Connectivity and Infrastructure Limitations

Many demonstrations and field-style use cases benefit from local execution. The final project can run on the same computer or local network after dependencies are installed.

2.2 Project Scope

The final runnable scope is a Flask and TensorFlow symptom classifier. Apache Spark EHR validation, native Android deployment, encryption, and regulated clinical validation remain future enhancements, not completed features of the current GitHub demo.

3 Proposed Solution

3.1 System Overview

The proposed system accepts a user's symptoms, normalizes the text, matches known phrases and aliases against the final symptom vocabulary, builds a binary feature vector, and sends it through the saved TensorFlow model. The application returns a primary diagnosis, top alternative predictions, confidence score, matched symptoms, urgency level, and suggested action.

3.1.1 Machine Learning Diagnosis Module

The final model is a neural network classifier trained on 137 binary symptom features and 42 disease classes.

3.1.2 Web Application Module

The Flask app provides the interface and API. It supports text input, browser voice input where available, diagnosis display, feedback logging, and print-friendly reports.

3.1.3 Offline and Local Network Operation

The application can run locally through python app.py or by using run_offline.bat on Windows. The app prints a local-network URL for another device connected to the same hotspot or Wi-Fi.

3.1.4 Honest Future Scope

The earlier project plan discussed Spark, Android, and EHR validation. These remain valuable future additions, but the final submitted implementation focuses on the working symptom classifier and Flask web app.

3.2 System Architecture

Figure 1 shows the final implemented architecture. The flow is intentionally compact so that the project is easy to run, inspect, and present from the GitHub repository.

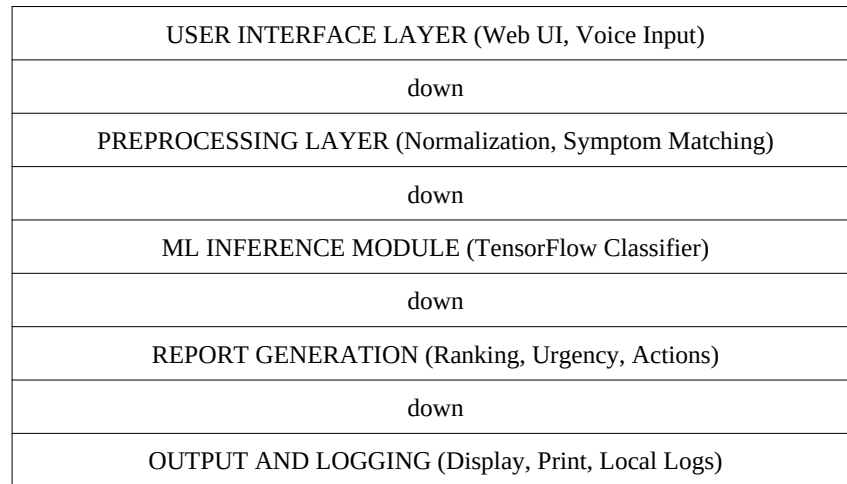


Figure 1. System Architecture Diagram

The backend loads three core artifacts from BACKEND/models: ml_model.h5, label_encoder.pkl, and symptom_vocabulary.json. The same vocabulary is used by the evaluation script and the web application, preventing feature-order mismatch during inference.

4 Technical Implementation

4.1 Technology Stack

Layer	Technology	Purpose
Backend	Python 3, Flask	API routes, model loading, prediction logic
Machine Learning	TensorFlow, scikit-learn	Training, inference, label encoding, metrics
Data Handling	Pandas, NumPy	CSV processing and binary feature matrices
Frontend	HTML5, CSS3, JavaScript	Symptom input, result rendering, print report
Storage	JSON, H5, PKL, CSV	Model artifacts, vocabulary, cleaned datasets
Documentation	Markdown and PDF	README, model card, final report

Table 1. Technology Stack Overview

4.2 Application Modules

The implementation is organized into reusable project files. `scripts/clean_datasets.py` creates the cleaned dataset. `scripts/train_model.py` trains and saves the model artifacts. `scripts/evaluate_model.py` evaluates the saved model. `WEB-APP/app.py` serves the application and prediction API.

4.3 Symptom Matching

The Flask backend normalizes user text by lowercasing, removing punctuation, and collapsing spaces. It checks exact symptom phrases from the vocabulary and also applies a small alias dictionary for common descriptions such as fever, chest pain, runny nose, and shortness of breath.

5 Data and Methods

5.1 Dataset Description

The dataset folder contains several symptom-to-disease CSV files. The final project uses the app-compatible sources and writes a cleaned pair of training and testing files under DATASETS/cleaned. Large unrelated raw datasets are intentionally ignored for the default GitHub demo.

Item	Final Value
Sources loaded	Training.csv, Testing.csv, dataset.csv, dataset (1).csv, cleaned_dataset.csv
Records before final cleaning	10502 loaded records before duplicate removal and filtering
Training rows	488
Testing rows	123
Disease classes	42
Binary symptom features	137
Important cleanup	Placeholder symptom values such as 0 are removed

Table 2. Final Cleaned Dataset Summary

5.2 Data Preprocessing

- Disease labels are normalized to consistent names such as COVID-19, GERD, Diabetes, and Hypertension.
- Symptom names are normalized into lowercase underscore-separated feature names.
- Duplicate disease-symptom records are removed.
- Classes with fewer than two samples are removed before model training.
- The final output files are Training_cleaned.csv, Testing_cleaned.csv, and cleaning_summary.json.

5.3 Model Training

The cleaned training file is split into training and validation subsets. The model uses two dense hidden layers with dropout and a 42-class softmax output layer. The label encoder is saved with the model so predictions can be converted back into disease names.

Raw CSV Sources	Clean Labels and Symptoms	Remove Duplicates	Train/Test Split	Save Model Artifacts
-----------------	---------------------------	-------------------	------------------	----------------------

Final cleaned dataset: 488 training rows, 123 testing rows, 42 classes, 137 symptom features

Figure 2. Dataset Cleaning and Training Pipeline

The final training run used 390 training samples and 98 validation samples from Training_cleaned.csv. The saved model artifacts were written to BACKEND/models.

6 Results and Discussions

6.1 Model Performance

The saved model was evaluated using `scripts/evaluate_model.py` on `DATASETS/cleaned/Testing_cleaned.csv`. The test file contains clean classroom-style symptom records, so the metrics are useful for verifying the project pipeline but should not be treated as clinical proof.

Metric	Value
Training accuracy	0.9949
Validation accuracy	0.9898
Testing samples	123
Testing classes	42
Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F1-score	1.0000

Table 3. Model Performance Metrics

The evaluation script reported no mismatches on the cleaned test file. However, the application disclaimer states that all outputs are educational preliminary suggestions and require professional confirmation.

6.2 Interface Results

The local browser test confirmed that the application loads the final model with 42 diseases and 137 symptoms. A respiratory example with high fever, cough, chest pain, chills, fatigue, phlegm, and breathlessness produced a ranked output with Pneumonia as the primary prediction and an Urgent label.

6.3 Web Application Interface Workflow

Step	User/System Action
1	User enters symptoms by text or voice input.
2	Frontend sends symptom text to the Flask /diagnose endpoint.
3	Backend matches symptom phrases and creates the binary model vector.
4	TensorFlow model returns class probabilities.
5	Interface displays diagnosis, confidence, urgency, evidence, and alternatives.

Figure 3. Web Application Interface Workflow

6.4 Diagnostic Report Sample

Report Field	Sample Output from Final App Test
Input	High fever, cough, chest pain, chills, fatigue, phlegm and breathlessness
Primary Diagnosis	Pneumonia
Confidence	36.5 percent
Urgency	Urgent
Matched Evidence	7 symptoms
Alternative Diagnoses	Bronchial Asthma and Tuberculosis
Recommended Action	Seek medical attention as soon as possible if symptoms are severe or worsening.

Figure 4. Sample Diagnostic Report Output

7 System Features and Capabilities

- Symptom-based disease prediction using a saved TensorFlow model.
- Final vocabulary of 137 symptom features and 42 disease classes.
- Top three ranked predictions with confidence scores.
- Matched symptom evidence count and readable symptom labels.
- Rule-based urgency labels: Urgent, Needs Review, Moderate, and Routine.
- Recommended action text for educational triage support.
- Browser voice input support where SpeechRecognition is available.
- Printable diagnosis report from the web interface.
- Offline/local-network launch through run_offline.bat after dependencies are installed.

These features make the project suitable for a classroom demonstration because the complete flow can be shown from data cleaning to model inference and interface output.

8 Development and Testing

8.1 Development Phases

- Phase 1: Organize project files and identify the runnable implementation gap.
- Phase 2: Add a real Flask application outside notebooks.
- Phase 3: Clean datasets and align model vocabulary with the web app.
- Phase 4: Retrain and save TensorFlow model artifacts.
- Phase 5: Improve interface language, offline running, and print report output.
- Phase 6: Finalize README, model card, report, and GitHub repository.

8.2 Testing Strategy

- Run dataset cleaner and inspect `cleaning_summary.json`.
- Retrain model and verify `training_summary.json`.
- Run saved-model evaluation on the cleaned test file.
- Check `/api/health` for loaded disease and symptom counts.
- Submit a sample symptom payload to `/diagnose` and inspect JSON output.
- Open the local web app in a browser and verify result rendering.

8.3 Testing Results

The final verification showed that the Flask app loaded 42 diseases and 137 symptoms. The browser test displayed the diagnosis panel successfully, and the API returned matched symptoms, ranked predictions, urgency, and recommended action.

9 Limitations and Future Enhancements

9.1 Current Limitations

- The model uses symptom features only and does not use laboratory results, imaging, vitals, or patient history.
- The test set is clean and small, so perfect test metrics do not prove real-world medical accuracy.
- Confidence scores are softmax probabilities, not calibrated medical certainty.
- The phrase-matching approach may miss vague, misspelled, or uncommon symptom descriptions.
- The system is not externally validated with real hospital workflows or regulated clinical datasets.

9.2 Completed and Planned Scope

Area	Current Status
Flask web app	Completed and runnable
TensorFlow symptom classifier	Completed and saved
Dataset cleaning pipeline	Completed for project-compatible symptom CSV files
Offline/local-network demo	Completed after dependencies are installed
Spark EHR validation	Planned future enhancement
Native Android app	Planned future enhancement
Clinical validation and certification	Future research and regulatory work

Table 4. Completed and Planned Scope

9.3 Future Enhancements

- Integrate larger and more diverse datasets after proper access and ethics review.
- Add Spark-based batch validation as a separate module when EHR-style data is available.
- Improve natural language symptom extraction beyond exact phrase matching.
- Calibrate confidence scores and test on noisy user-entered symptoms.
- Add multilingual symptom support for Indian languages.

10 Conclusion

The ML-Powered Medical Diagnosis Assistant successfully demonstrates a complete educational machine learning workflow for symptom-based preliminary diagnosis. The final version includes dataset cleaning, model training, saved artifacts, a Flask web application, ranked diagnosis output, urgency labels, matched evidence, printable reports, and GitHub-ready documentation.

The most important improvement in the final version is the alignment between the implementation and the report. Instead of claiming unimplemented Spark, Android, or clinical EHR modules as completed, the report presents them honestly as future work. This makes the project stronger, clearer, and more credible for academic evaluation.

The repository is available at <https://github.com/Rachith000bharadwaj/Machine-Learning>. The local project can be run through `run_offline.bat` or by starting the Flask application manually from `WEB-APP/app.py`.

References

- [1] TensorFlow Documentation. TensorFlow: Machine Learning Platform. <https://www.tensorflow.org/>
- [2] scikit-learn Documentation. Machine Learning in Python. <https://scikit-learn.org/>
- [3] Flask Documentation. Web Development Microframework. <https://flask.palletsprojects.com/>
- [4] Pandas Documentation. Python Data Analysis Library. <https://pandas.pydata.org/>
- [5] NumPy Documentation. Scientific Computing with Python. <https://numpy.org/>
- [6] Project symptom-to-disease CSV files included in the DATASETS folder.
- [7] Rachith000bharadwaj/Machine-Learning GitHub Repository.
<https://github.com/Rachith000bharadwaj/Machine-Learning>